

Professional Self-Assessment

Samuel Raymond Kwibe

CS 499 Computer Science Capstone

Southern New Hampshire University

Introduction

Completing the Computer Science program at Southern New Hampshire University has been a long process of turning a general interest in technology into a specific, demonstrable set of skills. When I started this program, I could write code that worked, but I did not yet understand what separated code that works from code that is professional, secure, maintainable, and genuinely useful to the people who depend on it. This capstone project, built around the MunchiesSNHU Food Waste Tracker, has been the place where all of the individual lessons from my coursework finally came together into a single coherent body of work. This self-assessment is my introduction to that work, and to who I am as a developer at the end of this program.

Collaborating in a Team Environment

Throughout this program, I learned that software is rarely built by one person working in isolation, even when it feels that way. In courses like CS 250, Software Development Lifecycle, I was introduced to the idea that software projects move through phases that involve different people with different priorities — requirements gathering, design, implementation, testing, and deployment — and that communication between those roles is often what determines whether a project succeeds or fails. This capstone gave me a very direct experience of that. While working through the three enhancement milestones, I was not writing every line of code myself. Instead, I had to take the requirements from my code review and translate them into clear, specific instructions for a developer, then review the work that came back, ask follow-up questions, and adjust the plan when something had already been completed differently than expected. At one point during Milestone Three, I learned that several of the algorithmic enhancements I had planned had already been implemented in a previous pass, and I had to quickly re-evaluate the remaining scope of work and communicate an updated plan. That kind of real-time adjustment

— staying organized while the plan changes underneath you — is exactly the kind of collaborative skill that matters in a professional team environment, where requirements shift and team members need to stay aligned without losing momentum.

Communicating with Stakeholders

Communication has been one of the most consistently emphasized skills throughout this program, and for good reason. A brilliant technical solution that nobody can understand or trust is not actually useful. In this capstone, I practiced communicating with two very different audiences. The code review video in Milestone One required me to explain the existing MunchiesSNHU codebase to an audience of peers and an instructor — walking through real files, pointing out specific weaknesses, and explaining my reasoning in plain language without burying the explanation in jargon. The three milestone narratives required a different kind of communication — written reflection that connects technical decisions to course outcomes and explains not just what was built, but why it was built that way and what trade-offs were considered. Writing the narrative for Milestone Three, where I explained why I chose a transparent weighted-scoring algorithm over a machine learning approach for the bin priority feature, taught me that good technical communication often means explaining what you did not do and why, not just what you did. In a professional setting, being able to clearly justify a design decision to a non-technical stakeholder — explaining why a simpler, more explainable solution was chosen over a more complex one — is a skill that directly affects whether stakeholders trust the technology they are relying on.

Data Structures and Algorithms

My foundation in data structures and algorithms came largely from CS 260, where I first worked with arrays, linked lists, trees, and sorting algorithms in a more formal way, learning to analyze the time and space complexity of different approaches. For a long time, those concepts felt somewhat theoretical — useful for passing exams but disconnected from the kind of work I imagined doing day to day. Milestone Three of this capstone changed that completely. Building the bin priority algorithm for the MunchiesSNHU project required me to take raw sensor data —

bin fullness, gas levels, pH readings, compost weight, and timestamps — and design a weighted scoring system that could rank an array of bins from highest to lowest priority. That is a sorting and data transformation problem, but it is also a design problem, because the weights I assigned to each factor represent real decisions about what matters most in a waste management context. I also worked extensively with MongoDB's aggregation pipeline, which is its own kind of algorithmic thinking — designing a sequence of stages that transform a large collection of records into grouped, summarized results. Between the priority algorithm and the aggregation pipelines, I gained hands-on experience with exactly the kind of algorithmic problem-solving that CS 260 introduced conceptually, but this time applied to a real application with real data.

Software Engineering and Database Design

Software engineering and database design are the two areas where I feel I grew the most over the course of this program, and Milestones Two and Four of this capstone are the clearest evidence of that growth. In Milestone Two, I worked through the MunchiesSNHU backend and found that while the project had a recognizable folder structure with routes, controllers, and models, several of those pieces were not actually connected. Some controllers had working logic with no routes pointing to them. Business logic was sitting inside route files instead of controllers. A token-generation function was duplicated in two places. Fixing these issues required applying the MVC architecture principles from IT 145 and IT 315 in a real codebase rather than a textbook example, and it taught me that a project can look organized on the surface while still having structural problems underneath. In Milestone Four, I turned to the database layer itself, building on the SQL foundations from DAD 220 but applied to MongoDB instead. I verified that the schema already enforced numeric range validation on critical fields, added missing indexes to support the application's most common queries, removed duplicate settings models that could have caused data to fall out of sync, and discovered and removed an exposed environment file that contained database credentials sitting in a publicly accessible folder. That last discovery in particular reinforced something I had learned more abstractly in earlier courses — that a database is only as

trustworthy as the access controls and credential management around it, regardless of how well the schema itself is designed.

Security

Security has been a thread running through every part of this capstone rather than something addressed in isolation. In Milestone Two, I activated security middleware — including rate limiting, request sanitization, and secure HTTP headers — that had been installed in the project but never turned on, and I unified the authentication system so that both signup and login relied on the same JWT-based flow protected by middleware that checks both authentication and role-based authorization. In Milestone Three, the defensive data handling I added was as much a security practice as an algorithmic one, since an application that crashes or behaves unpredictably when it receives unexpected input is a security risk, not just a reliability one. In Milestone Four, the most significant security work was finding and removing an exposed .env file containing database credentials from a publicly accessible folder — a vulnerability that had existed in the project for some time without being noticed. That discovery was a meaningful lesson for me. It is easy to think of security as something you design in from the start and then move on from, but real security work also involves continuously auditing existing code for things that were missed, regardless of how long ago they were introduced.

How My Artifacts Fit Together

All three enhancements in this ePortfolio were built on a single artifact — the MunchiesSNHU Food Waste Tracker — and I made that choice deliberately rather than using three separate projects. I wanted my ePortfolio to demonstrate not just three isolated skills, but how those skills build on each other in a real application over time. The before-enhancement snapshot of this project shows a full-stack application with good intentions but real structural, algorithmic, and database problems — broken routes, mismatched field names between frontend and backend, an inactive security layer, a placeholder-only algorithm, and an exposed credentials file. The Software Design and Engineering enhancement in Milestone Two addressed the structural foundation, connecting the pieces that were never wired together and activating the security

protections that were already present but dormant. The Algorithms and Data Structures enhancement in Milestone Three built directly on that foundation, adding the bin priority algorithm and reporting aggregations that turn the application's collected data into something actionable. The Databases enhancement in Milestone Four completed the picture by hardening the data layer itself — enforcing validation, optimizing queries with indexes, and closing a real security gap. Viewed together, these three enhancements tell the story of an application moving from partially functional to fully connected, from passive data collection to active decision support, and from structurally sound to genuinely secure.

Closing

I am entering the job market with a portfolio that I did not just build — I can explain every decision in it, including the ones I considered and chose not to make. That ability to reason through trade-offs, communicate them clearly, and back them up with working code is, I believe, exactly what employers are looking for in an entry-level developer who is ready to contribute to a real team. The sections that follow this self-assessment present the code review, the original and enhanced versions of the MunchiesSNHU Food Waste Tracker, and the narratives that walk through each enhancement in detail.

AI Usage Disclosure

An AI writing assistant was used to help organize and draft portions of this self-assessment. All content was reviewed, edited, and rewritten to reflect my own understanding, experiences, and growth throughout the Computer Science program.